

Hands-on Python Tutorial for DRL

Module 4

Continuous Control, DDPG, and PPO

This tutorial is a practical companion to Module 4. Its purpose is to help students understand continuous-control actor-critic methods by implementing them in Python and then evaluating what happens when key design choices are changed.

Prepared for the DRL Course

March 2026

How this tutorial matches Module 4

Module 4 moves from the actor–critic bridge to continuous control, then introduces two main families: DDPG and PPO. This tutorial follows the same logic. Students first inspect a continuous-control environment, then implement the core pieces of DDPG, then implement the core pieces of PPO, and finally compare the two methods through evaluation.

Learning goal. By the end of this tutorial, students should be able to connect each main idea in Module 4 to concrete Python objects: actor networks, critic networks, replay buffers, target networks, rollout buffers, advantage estimation, and clipped policy updates.

1. How students should use the starter code

The file `module4_tutorial_starter.py` is not meant to be read from top to bottom and then executed blindly. Students should use it as a guided scaffold and work through it in a fixed order.

1.1 What the starter code already provides

The starter file already contains the overall training structure, the network class definitions, the data buffers, the evaluation functions, and the plotting helper. This means students do *not* need to design the whole project architecture from scratch. Instead, they should focus on understanding and verifying the algorithmic core.

1.2 What students are expected to do

Students should do the following with the starter code:

1. Run the file once in its initial form to inspect the environment output in the main block.
2. Read the DDPG part first and match each key line to the DDPG equations from the lecture.
3. Read the PPO part second and match each key line to the PPO and GAE equations from the lecture.
4. Use the commented training calls in the main block to run *one method at a time*.
5. Modify hyperparameters and compare the resulting learning curves.
6. Write a short analysis explaining what each algorithm is doing and why the performance changes across experiments.

1.3 How to work through the file in practice

A good workflow is:

1. Open the file and identify its five main parts: helper functions, DDPG components, PPO components, training loops, and the main block.
2. Start from the main block at the bottom so that the student sees how the file is intended to be executed.
3. Move upward to inspect the functions that are called by the main block.
4. For every block marked with `TODO`, the student should pause and explain in words what that line is computing before running the code.
5. After understanding one block, the student should run a short experiment and verify that the output makes sense.

Important expectation

Students should **not** treat the starter code as a black box. Even if a function already looks complete, the task is to understand what information enters that function, what quantity is computed inside it, and how the result affects the learning update.

1.4 TODO map inside the starter file

TODO	Location in code	What the student should understand or verify
1–2	<code>ddpg_update()</code>	How the target actor and target critic produce the bootstrapped DDPG target
3	<code>ddpg_update()</code>	How the online critic predicts $Q(s, a)$ for sampled replay transitions
4	<code>ddpg_update()</code>	Why the actor loss is the negative critic score on the actor's own actions
5	<code>ddpg_update()</code>	Why target networks are updated slowly instead of copied aggressively
6–7	GAE function (<code>compute_gae_and_returns</code>)	How TD residuals become GAE advantages and returns
8–9	PPO update (<code>ppo_update</code>)	How PPO recomputes the probability ratio and applies clipping
10	PPO update (<code>ppo_update</code>)	How the value critic fits returns with a regression loss

1.5 What students should change and what they should leave unchanged

Students should mainly change:

- the commented training calls in the main block,
- hyperparameters such as `noise_std`, `tau`, `clip_eps`, and `lam`,
- plotting and experiment settings,
- any optional extra comments or print statements they add for debugging.

Students should usually leave the overall class and function structure unchanged unless they want to perform an extension study.

1.6 Minimum sequence for a successful student workflow

1. Run the environment inspection block.
2. Run DDPG only and confirm that evaluation returns improve.
3. Run one DDPG ablation, such as changing the exploration noise or τ .
4. Run PPO only and confirm that evaluation returns improve.
5. Run one PPO ablation, such as changing ϵ or λ .
6. Put the DDPG and PPO learning curves on comparable plots.
7. Explain the differences in words.

2. Tutorial design choices

2.1 Environment

We use Pendulum-v1 from `gymnasium`. It is a standard continuous-control benchmark with:

- a continuous action in a bounded interval,
- a low-dimensional state,
- fast training compared with larger physics simulators.

This keeps the focus on the algorithm rather than environment engineering.

2.2 Library stack

The tutorial assumes Python 3.10+ with:

- `gymnasium[classic-control]`
- `torch`
- `numpy`
- `matplotlib`

2.3 Suggested installation

```
pip install gymnasium[classic-control] torch numpy matplotlib
```

2.4 Why Pendulum?

In Module 4, one of the key conceptual shifts is that the action is not selected from a short list. Pendulum makes that visible immediately: the action is a real-valued torque, so the student cannot rely on a DQN-style $\arg \max_a Q(s, a)$ over a few discrete actions.

3. Part 0: inspect the continuous-control environment

Before implementing any RL method, students should inspect what the environment returns.

```
import gymnasium as gym

env = gym.make("Pendulum-v1")
state, info = env.reset(seed=42)
print("Initial state:", state)
print("State dimension:", env.observation_space.shape[0])
print("Action dimension:", env.action_space.shape[0])
print("Action low:", env.action_space.low)
print("Action high:", env.action_space.high)

random_action = env.action_space.sample()
next_state, reward, terminated, truncated, info = env.step(random_action)
print("Random action:", random_action)
print("Next state:", next_state)
print("Reward:", reward)
print("Done flag:", terminated or truncated)
```

Quick checkpoint

Students should be able to identify the transition tuple

$$(s_t, a_t, r_{t+1}, s_{t+1}, \text{done}_t)$$

and explain why a_t is now a real-valued vector rather than an integer action label.

Hint

Ask students to print five random actions. They should notice that the values are continuous and bounded. This is the direct reason why Module 4 moves beyond DQN-style action maximization.

4. Part I: actor–critic building blocks before DDPG and PPO

4.1 Two actor choices

In code, the actor can be implemented in two different ways:

- **Deterministic actor:** the network outputs one action directly. This is the DDPG case.
- **Stochastic actor:** the network outputs the parameters of a probability distribution. This is the PPO case.

4.2 Two critic choices

Likewise, the critic can take two forms:

- **Action-value critic** $Q_\phi(s, a)$, used by DDPG.
- **State-value critic** $V_\psi(s)$, used by PPO.

4.3 Concept-to-code map

Concept	Python object or operation
Deterministic actor	<code>actor(state) -> action</code>
Stochastic actor	<code>actor(state) -> mean, std</code>
Action-value critic	<code>critic(state, action) -> scalar</code>
State-value critic	<code>state_net(state) -> scalar</code>
Replay memory	Stores old transitions for DDPG
Rollout buffer	Stores fresh trajectories for PPO
Target networks	Slow-moving copies used by DDPG
Advantage estimate	Computed from TD residuals or GAE for PPO
Clipped policy ratio	PPO safety mechanism in the actor loss

5. Part II: DDPG tutorial

5.1 Why DDPG is a good first continuous-control implementation

DDPG is a natural continuation of the value-based ideas from earlier modules. It still uses a critic and still bootstraps, but instead of maximizing over a discrete action set, it learns an actor that outputs the continuous action directly.

5.2 DDPG idea in one line

$$a_t = \mu_{\theta}(s_t) + \varepsilon_t, \quad y_t = r_{t+1} + \gamma(1 - \text{done}_t)Q_{\phi^{-}}(s_{t+1}, \mu_{\theta^{-}}(s_{t+1})).$$

The first equation is the data-collection rule. The second is the critic target.

5.3 Step 1: define the actor and critic networks

The actor maps state to action. The critic maps state-action pairs to a scalar value.

```
class DDPGActor(nn.Module):
    def __init__(self, state_dim, action_dim, action_limit):
        super().__init__()
        self.action_limit = action_limit
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128), nn.ReLU(),
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, action_dim), nn.Tanh()
        )

    def forward(self, state):
        return self.action_limit * self.net(state)

class DDPGCritic(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim + action_dim, 128), nn.ReLU(),
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, state, action):
        x = torch.cat([state, action], dim=-1)
        return self.net(x)
```

The `tanh` output is useful because the action must stay within the environment bounds. The action is then scaled by the action limit.

5.4 Step 2: build the replay buffer

DDPG is off-policy, so it does not need to throw away old transitions immediately.

```
replay_buffer.add(state, action, reward, next_state, done)
states, actions, rewards, next_states, dones = replay_buffer.sample(batch_size)
```

Hint

Students should verify the tensor shapes after sampling. A very common implementation error is a silent shape mismatch between `(batch, action_dim)` and `(batch,)`.

5.5 Step 3: add exploration noise

A deterministic actor does not explore by itself. During interaction, students should use

$$a_t = \mu_\theta(s_t) + \varepsilon_t,$$

then clip the result into the valid action interval.

```
with torch.no_grad():
    action = actor(state_t).cpu().numpy()[0]
    noise = noise_std * np.random.randn(action_dim)
    behavior_action = np.clip(action + noise, action_low, action_high)
```

Common bug

Do not add noise during evaluation. Training should use noisy actions, but evaluation should use the deterministic actor output alone.

5.6 Step 4: compute the DDPG critic target

For a sampled mini-batch, students should implement:

```
with torch.no_grad():
    next_actions = target_actor(next_states)
    target_q = target_critic(next_states, next_actions)
    y = rewards + gamma * (1.0 - dones) * target_q
```

This is the exact code version of the DDPG target equation.

5.7 Step 5: update the critic

The online critic should fit the target using mean-squared error.

```
current_q = critic(states, actions)
critic_loss = F.mse_loss(current_q, y)
critic_optimizer.zero_grad()
critic_loss.backward()
critic_optimizer.step()
```

5.8 Step 6: update the actor

The actor should choose actions that receive large critic values. In code, the student usually minimizes the negative critic score:

```
actor_loss = -critic(states, actor(states)).mean()
actor_optimizer.zero_grad()
actor_loss.backward()
actor_optimizer.step()
```

What the actor update means

There is no supervised label saying “the correct torque is 0.31.” The actor changes because the critic assigns larger or smaller values to the actions it proposes.

5.9 Step 7: soft-update the target networks

After updating the online networks, the target networks should move slowly:

```
for p, p_targ in zip(actor.parameters(), target_actor.parameters()):
```

```
p_targ.data.mul_(1 - tau)
p_targ.data.add_(tau * p.data)
```

The same idea is applied to the critic and target critic.

5.10 Step 8: write the DDPG training loop

Students should combine the previous parts into the following rhythm:

1. reset environment,
2. collect a noisy action,
3. store the transition,
4. once enough samples exist, update critic then actor,
5. soft-update both target networks,
6. repeat.

5.11 How students should use the starter code for DDPG

Inside the starter file, students should work with DDPG in this exact order:

1. Read `ReplayBuffer`, `DDPGActor`, and `DDPGCritic` first.
2. Open `ddpg_update()` and explain TODO 1–5 line by line.
3. Open `train_ddpg()` and identify where actions are sampled, where replay memory is filled, and where updates begin.
4. In the main block, uncomment only the DDPG training call first.
5. Run a short experiment, for example `total_steps=20000`, and inspect whether the printed evaluation return improves over time.
6. Only after the first run works should the student start changing `noise_std` or `tau`.

Hint

A good debugging strategy is to print tensor shapes in `ddpg_update()` during the first run. The student should verify that the critic target, current critic output, rewards, and done flags all have compatible batch dimensions.

5.12 Step 9: evaluate DDPG properly

Every fixed number of environment steps, students should run a small evaluation loop with *no exploration noise*. A simple metric is the mean episode return over 5 or 10 evaluation episodes.

```
def evaluate_ddpg(env, actor, episodes=5):
    returns = []
    for _ in range(episodes):
        state, _ = env.reset()
        done = False
        total_reward = 0.0
        while not done:
            with torch.no_grad():
                action = actor(torch.tensor(state, dtype=torch.float32).unsqueeze
(0))
            action = action.cpu().numpy()[0]
```



```

        state, reward, terminated, truncated, _ = env.step(action)
        total_reward += reward
        done = terminated or truncated
        returns.append(total_reward)
    return np.mean(returns)

```

5.13 What to plot for DDPG

Students should at minimum plot:

- evaluation return vs environment steps,
- critic loss vs update step,
- actor loss vs update step.

These plots help separate “the code runs” from “the algorithm is actually learning.”

6. Part III: PPO tutorial

6.1 Why PPO feels different from DDPG

PPO is still actor–critic, but almost every engineering choice changes:

- the actor is stochastic rather than deterministic,
- the critic is $V(s)$ rather than $Q(s, a)$,
- data comes from fresh rollouts rather than a long replay memory,
- the policy update is constrained by clipping.

6.2 PPO idea in one line

$$L^{\text{clip}}(\theta) = \mathbb{E} \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

Students do not need to memorize this first. They need to recognize where each term comes from in code.

6.3 Step 1: define the Gaussian actor and value critic

For continuous-action PPO, a common design is to let the actor output the mean and use a trainable log-standard deviation.

```

class GaussianActor(nn.Module):
    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128), nn.Tanh(),
            nn.Linear(128, 128), nn.Tanh(),
            nn.Linear(128, action_dim)
        )
        self.log_std = nn.Parameter(torch.zeros(action_dim))

    def forward(self, state):
        mean = self.net(state)
        std = torch.exp(self.log_std).expand_as(mean)
        return mean, std

```

```

class ValueCritic(nn.Module):
    def __init__(self, state_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128), nn.Tanh(),
            nn.Linear(128, 128), nn.Tanh(),
            nn.Linear(128, 1)
        )

    def forward(self, state):
        return self.net(state)

```

6.4 Step 2: collect a rollout buffer

For each interaction step, students should store:

- state,
- sampled action,
- reward,
- done flag,
- old log-probability of the sampled action,
- critic value estimate.

This buffer is used only for the current update round.

6.5 Step 3: compute TD residuals and GAE

After a rollout is collected, students should compute the TD residuals

$$\delta_t = r_{t+1} + \gamma(1 - \text{done}_t)V_\psi(s_{t+1}) - V_\psi(s_t),$$

then compute advantages backward through time with GAE.

```

advantages = np.zeros_like(rewards)
last_gae = 0.0
for t in reversed(range(len(rewards))):
    delta = rewards[t] + gamma * (1.0 - dones[t]) * next_values[t] - values[t]
    last_gae = delta + gamma * lam * (1.0 - dones[t]) * last_gae
    advantages[t] = last_gae
returns = advantages + values

```

Hint

Students should normalize the advantages before PPO updates. This often improves optimization stability:

$$\hat{A} \leftarrow \frac{\hat{A} - \text{mean}(\hat{A})}{\text{std}(\hat{A}) + 10^{-8}}.$$

6.6 Step 4: recompute probabilities under the new actor

During PPO updates, the student should run the current actor on the stored batch, obtain the new log-probabilities, and compute the ratio

$$\rho_t(\theta) = \exp(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{old}}(a_t|s_t)).$$

6.7 Step 5: implement the clipped actor loss

```
ratio = torch.exp(new_log_probs - old_log_probs)
surr1 = ratio * advantages
surr2 = torch.clamp(ratio, 1 - clip_eps, 1 + clip_eps) * advantages
actor_loss = -torch.min(surr1, surr2).mean()
```

This is the most important PPO line in the whole tutorial.

6.8 Step 6: fit the value critic

The value critic is trained with a regression loss toward the computed returns:

```
value_pred = critic(states).squeeze(-1)
value_loss = F.mse_loss(value_pred, returns)
```

Optionally, students can add an entropy bonus to encourage exploration.

6.9 Step 7: perform multiple PPO epochs on the same rollout

Unlike DDPG, PPO usually takes several gradient steps on the same fresh batch before collecting more data. Students should:

1. shuffle the rollout indices,
2. split them into mini-batches,
3. update actor and critic for several epochs,
4. then discard the rollout and collect fresh data.

6.10 How students should use the starter code for PPO

Inside the starter file, students should work with PPO in this order:

1. Read `GaussianActor` and `ValueCritic` first.
2. Study `collect_ppo_rollout()` and identify exactly what is stored in the rollout buffer.
3. Open `compute_gae_and_returns()` and explain TODO 6–7 using the GAE formula from the lecture.
4. Open `ppo_update()` and explain TODO 8–10 using the PPO clipped objective and value loss.
5. In the main block, uncomment only the PPO training call.
6. Run a short experiment and inspect whether the evaluation return improves after each rollout-update cycle.
7. Then change `clip_eps` or `lam` for one controlled comparison.

Hint

Students should verify that `old_log_probs` are stored during rollout collection and that `new_log_probs` are recomputed during optimization. This is one of the most important conceptual differences in PPO.

6.11 Step 8: evaluate PPO

Evaluation should usually use the actor mean rather than a noisy sampled action. The purpose is to measure the quality of the learned controller, not the amount of exploration noise.

6.12 What to plot for PPO

Students should at minimum plot:

- evaluation return vs environment steps,
- actor loss vs update iteration,
- value loss vs update iteration,
- average episode length if desired.

7. Part IV: guided comparison between DDPG and PPO

Students should not stop after “both codes run.” The learning objective of Module 4 is to understand the design tradeoffs.

7.1 Comparison checklist

Aspect	DDPG question	PPO question
Exploration	What happens when the noise scale is too small or too large?	What happens when the policy std starts very small or very large?
Data reuse	Does replay improve sample efficiency?	How many epochs on one rollout are still helpful?
Stability	How sensitive is learning to target update rate τ ?	How sensitive is learning to ϵ in clipping?
Critic role	Does a poor Q-critic mislead the actor?	Do poor value estimates lead to noisy advantages?
Outcome	Does the deterministic controller become smooth?	Does the clipped update make learning less erratic?

7.2 Recommended medium-difficulty experiments

1. **DDPG exploration study:** compare two or three noise scales and plot evaluation return.
2. **DDPG target-update study:** compare a smaller and larger τ .
3. **PPO clipping study:** compare $\epsilon = 0.1$ and $\epsilon = 0.2$.
4. **PPO GAE study:** compare $\lambda = 0.95$ and $\lambda = 1.0$.
5. **Final comparison:** run DDPG and PPO under one chosen setting and compare learning curves on the same plot.

Hint

Students should run each setting with more than one random seed if time allows. A single RL run can be misleading.

8. Part V: step-by-step expectations for the student report

A good tutorial submission should include the following pieces.

8.1 Minimum deliverables

1. A short description of the environment and state/action spaces.
2. A description of the implemented actor and critic networks.
3. One paragraph explaining the DDPG critic target and actor loss in words.
4. One paragraph explaining the PPO ratio, advantage, and clipped objective in words.
5. Learning curves for both algorithms.
6. One small ablation or hyperparameter comparison for each algorithm.
7. A conclusion comparing DDPG and PPO on this task.

8.2 Questions students should answer in their own words

- Why is a replay buffer natural for DDPG but not for PPO?
- Why does DDPG need target versions of both actor and critic?
- Why does PPO compare the new policy with the old policy on the same sampled action?
- What does a positive advantage mean in PPO?
- Which method looked more stable in your experiments, and why do you think so?

9. Common implementation mistakes

Common bug

DDPG: forgetting to clip actions to environment bounds, bootstrapping through terminal states, updating the target networks too aggressively, or evaluating with exploration noise still turned on.

Common bug

PPO: forgetting to store old log-probabilities, mixing rollout data from different policies, not recomputing new log-probabilities during updates, or using returns where advantages should be used in the clipped actor loss.

Common bug

Both methods: using inconsistent tensor shapes, silently mixing NumPy arrays and torch tensors on different devices, and judging performance from one short training run.

10. Suggested workflow for the tutorial session

1. Inspect the environment and print transitions.
2. Implement and debug DDPG first.
3. Plot DDPG learning curves.
4. Implement and debug PPO second.
5. Plot PPO learning curves.
6. Run one or two controlled comparisons.
7. Write the short analysis answering the conceptual questions.

10.1 Exactly what the student should run

The bottom of the starter file contains three important lines:

```
# ddp_stats = train_ddpg(total_steps=20_000, noise_std=0.15, tau=0.005)
# ppo_stats = train_ppo(total_steps=20_480, clip_eps=0.2, lam=0.95)
# plot_training_curves(ddpg_stats=ddpg_stats, ppo_stats=ppo_stats)
```

Students should use them in this order:

1. Uncomment only the DDPG line and run it.
2. After DDPG finishes, either call `plot_training_curves(ddpg_stats=ddpg_stats)` or store the returned dictionary and plot later.
3. Comment DDPG again, then uncomment only the PPO line and run it.
4. Plot PPO separately, or run both again and use the combined plotting call.

Running both long training jobs simultaneously while still debugging is usually a poor strategy because it becomes harder to identify which part failed.

10.2 What evidence the student should collect

By the end of the tutorial, the student should have:

- one successful DDPG run,
- one successful PPO run,
- at least one hyperparameter comparison for each method,
- learning curves and a short written interpretation.

Final takeaway. DDPG and PPO are both actor-critic methods, but they solve the continuous-control problem in different ways. DDPG relies on a deterministic actor, replay memory, target networks, and external exploration noise. PPO relies on a stochastic actor, fresh rollouts, advantage estimation, and clipped policy updates. The point of this tutorial is not only to get code running, but to understand why these design choices exist.

Suggested reading

- Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., 2018.
- David Silver et al., “Deterministic Policy Gradient Algorithms,” ICML 2014.
- Timothy P. Lillicrap et al., “Continuous Control with Deep Reinforcement Learning,” 2015/2016.
- John Schulman et al., “Proximal Policy Optimization Algorithms,” 2017.
- OpenAI Spinning Up notes for DDPG and PPO.